



Technische
Universität
Braunschweig



25-June-24

igp



Project Proposal: MLRSNet Dataset

Disha Ghosh, Nizaanth Raja

MSc. Data Science, MSc. Computation Science

Deep Learning Course, IGP-TUBS, Dr. -Ing. Mehdi Maboudi, Dr. Pedro Achanccaray

Outline

- Data Generator Implementation
- Model Development
- Improving CNN Training
- Transfer Learning
- Model Evaluation

Quick Recap!

What is MLRSNet Dataset?

- The MLRSNet dataset is composed of high spatial resolution optical satellite images.
- It comprises over 109,161 remote - sensing images, each labeled with one or more of the 46 categories, and the number of sample images in a category varies from 1,500 to 3,000.
- The images have a fixed size of 256×256 pixels. Moreover, each image in the dataset is tagged with several of 60 predefined class labels, and the number of labels associated with each image varies from 1 to 13.
- These categories cover a wide range of natural and man-made objects, making the dataset versatile for various applications. This diversity helps in building robust models that can generalize well across different scenarios.

Source: [MLRSNet GitHub](#)

Data Generator Implementation

Source: [Github](#)

Key Components:

1. Constructor (init):

- Initialises the data generator with the following parameters:
 - path_images: Array of image file paths.
 - labels: Array of corresponding labels.
 - batch_size: Number of samples per batch.
 - n_classes: Number of classes for classification.
 - target_size: Target size for each image (default is 256x256).
 - shuffle: Whether to shuffle the data after each epoch (default is True).

2. **on_epoch_end()**: Shuffles the dataset after each epoch if shuffle is set to True.

3. **len()**: Returns the number of batches per epoch based on the batch size.

4. **__get_image(path_image)**: Reads an image from the given path, resizes it to the target size, converts it to RGB, expands dimensions for batching, and normalizes pixel values.

5. **__get_label(label)**: Converts the label to one-hot encoding.

6. **getitem(idx)**: Retrieves a batch of images and their corresponding labels, reads and preprocesses each image and label in the batch, returns a tuple of image and label arrays ready for training.

Model Development

Source: [Original Paper on MLRSNet dataset.](#)

•Vanilla CNN:

- Convolutional neural network with a custom number of layers.
- Serves as a benchmark architecture.

•VGGNet16:

- Deeper network with 16 layers.
- Known for simplicity and effectiveness.

•DenseNet201:

- Utilizes dense blocks to improve information flow.
- Deeper network with 201 layers.

Improving CNN Training

Source: [Original Paper on MLRSNet dataset.](#)

1. Regularization Techniques:

- Dropout:** Randomly drop neurons during training to prevent overfitting, depending on the accuracy of the model.
- Early Stopping:** Halt training when performance on the validation set stops improving (tentatively: Patience Level of 20).
- Batch Normalization:** Normalize activations to accelerate training and improve stability.

2. Learning Rate: 0.01

3. Optimiser: Adam

4. Batch Size: 32

Transfer Learning

Approach

•Pretrained Models:

- Utilize models pretrained on large datasets like ImageNet.
- Example: VGGNet16 and DenseNet201 as used in the [original research paper](#).

•Fine-Tuning:

- Start with pretrained weights.
- Retrain some or all layers on the MLRSNet dataset.

•Custom Architectures:

- Experiment with advanced architectures or modifications to existing models to enhance performance.

Model Evaluation

Source: [Original Paper on MLRSNet dataset.](#)

Metrics:

- **Mean Average Precision (mAP):** Measure of precision across different thresholds.
- **Average F1 Score:** Harmonic mean of precision and recall.

Expected Results

We anticipate that our custom CNN to perform on par if not better than the fine-tuned deep learning models , with the DenseNet201 expected to reach approximately 90% accuracy.

Thank You!
Open to Recommendations.